

MODULE 1 • DAY 1

Kubernetes Workloads

From Pods to CronJobs — the building blocks that run and manage every application on K3s

K3s Intermediate to Advanced • 15-Day Instructor-Led Programme



Today's Workload Roadmap

Twelve building blocks, in the order pods actually depend on each other

PO

Pods

RS

ReplicaSets

DP

Deployments

SS

StatefulSets

DS

DaemonSets

JB

Jobs

CJ

CronJobs

IC

Init Containers

SC

Sidecar Containers

LH

Lifecycle Hooks

SD

Startup/Shutdown

HO

Hands-on Labs



Pods

The smallest deployable unit in Kubernetes

3

KEY POINTS

- Wraps one or more containers that share network (IP) and storage
- Containers in a pod always co-scheduled on the same node
- Pods are ephemeral — never created directly in production
- Every higher workload (Deployment, Job, StatefulSet) ultimately manages pods

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
```

USE CASE quick debugging pods, one-off test containers



ReplicaSets

Keeps a stable number of identical pods running

4

KEY POINTS

- Continuously reconciles: watches actual vs. desired replica count
- Uses a label selector to find and adopt matching pods
- Replaces any pod that crashes, is deleted, or fails a probe
- Rarely created by hand — Deployments manage ReplicaSets for you

`replicaset.yaml`

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels: {app: nginx}
  template:
    metadata:
      labels: {app: nginx}
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

USE CASE understanding self-healing before adopting Deployments



Deployments

Declarative updates for pods, built on top of ReplicaSets

5

KEY POINTS

- Manages ReplicaSets to enable rolling updates and rollbacks
- Change the image/spec → old ReplicaSet scales down, new one up
- `kubectl rollout undo` instantly reverts to the previous revision
- The default choice for stateless application workloads

`deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels: {app: nginx}
  template:
    metadata:
      labels: {app: nginx}
    spec:
      containers:
        - name: nginx
          image: nginx:1.27

# kubectl rollout status deployment/nginx-deploy
```

USE CASE web apps, APIs, any stateless replicated service



StatefulSets

Ordered, identity-preserving workloads for stateful apps

6

KEY POINTS

- Each pod gets a stable name and network ID (mysql-0, mysql-1 ...)
- Pods are created, scaled, and deleted in strict order
- volumeClaimTemplates gives every replica its own persistent volume
- Pairs with a Headless Service for stable per-pod DNS

statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  selector:
    matchLabels: {app: mysql}
  template:
    metadata:
      labels: {app: mysql}
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
  volumeClaimTemplates:
    - metadata: {name: data}
```

USE CASE databases, message queues, any app needing stable identity



DaemonSets

Exactly one pod copy on every (or every matching) node

7

KEY POINTS

- Automatically adds a pod when a new node joins the cluster
- Automatically removes the pod when a node is drained/removed
- nodeSelector/affinity can target a subset of nodes only
- Bypasses normal scheduling — one instance per node is guaranteed

`daemonset.yaml`

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels: {app: node-exporter}
  template:
    metadata:
      labels: {app: node-exporter}
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter
          ports:
            - containerPort: 9100
```

USE CASE log shippers, node monitoring agents, CNI/CSI node plugins



Jobs

Run-to-completion workloads — not long-running services

8

KEY POINTS

- Creates one or more pods and tracks them until they succeed
- `backoffLimit` controls how many times a failed pod is retried
- `completions/parallelism` control batch and parallel execution
- Pod stays Completed after success — nothing further is scheduled

`job.yaml`

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  backoffLimit: 3
  template:
    spec:
      containers:
      - name: migrate
        image: myapp/migrate:1.0
        command: ["/migrate.sh"]
        restartPolicy: Never
```

USE CASE database migrations, batch processing, one-off scripts



CronJobs

Jobs on a schedule, using standard cron syntax

9

KEY POINTS

- Creates a new Job automatically at each scheduled time
- schedule field uses five-field cron syntax (min hr dom mon dow)
- concurrencyPolicy: Forbid/Replace controls overlapping runs
- successfulJobsHistoryLimit keeps recent run history for review

`cronjob.yaml`

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: myapp/backup:1.0
          restartPolicy: OnFailure
```

USE CASE nightly backups, report generation, scheduled cleanup tasks



Init Containers

Setup steps that must finish before the app starts

10

KEY POINTS

- Run one at a time, in order, before any app container starts
- Each must exit 0 (success) before the next one begins
- If an init container fails, Kubernetes restarts the pod
- Ideal for prerequisites the app container shouldn't own

pod-init.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-init
spec:
  initContainers:
    - name: wait-for-db
      image: busybox
      command: ["sh", "-c",
        "until nc -z db 5432; do sleep 2; done"]
  containers:
    - name: app
      image: myapp:1.0
```

USE CASE wait-for-dependency checks, config/secret fetch, schema setup



Sidecar Containers

Helper containers that run alongside the main app, for its whole life

11

KEY POINTS

- Share the pod's network, volumes, and lifecycle with the app
- Native sidecars (K8s 1.29+): `initContainers` + `restartPolicy: Always`
- Start before, and keep running after, the main container
- Kept separate so the app image stays focused on one job

`pod-sidecar.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  initContainers:
    - name: log-shipper
      image: fluent-bit:2.2
      restartPolicy: Always    # native sidecar
  containers:
    - name: app
      image: myapp:1.0
```

USE CASE log/metrics shippers, service-mesh proxies (Envoy), config sync



Lifecycle Hooks

Code that runs exactly at container start and stop

12

KEY POINTS

- postStart fires right after the container is created
- preStop fires right before the container receives SIGTERM
- Hooks run exec commands or httpGet/tcpSocket calls
- preStop is the standard place for graceful connection draining

lifecycle-hooks.yaml

```
containers:
- name: app
  image: myapp:1.0
  lifecycle:
    postStart:
      exec:
        command: ["sh", "-c",
                  "echo Started > /tmp/state"]
    preStop:
      exec:
        command: ["sh", "-c",
                  "sleep 10 && ./drain.sh"]
```

USE CASE warm-up caches on start, drain load balancer before stop



Startup & Shutdown Behaviour

How a pod proves it's ready, and how it exits without dropping traffic

13

STARTUP

- startupProbe delays liveness/readiness checks for slow-boot apps
- Pod only receives traffic once readinessProbe passes

SHUTDOWN SEQUENCE

- 1 Pod marked Terminating; removed from Service endpoints
- 2 preStop hook runs (if defined)
- 3 SIGTERM sent to the container's main process
- 4 Grace period elapses (terminationGracePeriodSeconds)
- 5 SIGKILL sent if the process hasn't exited yet

probe-and-grace.yaml

```
spec:
  terminationGracePeriodSeconds: 30
  containers:
  - name: app
    image: myapp:1.0
    startupProbe:
      httpGet: {path: /healthz, port: 8080}
      failureThreshold: 30
      periodSeconds: 2
    readinessProbe:
      httpGet: {path: /ready, port: 8080}
      periodSeconds: 5
```

1

Deploy a Pod & ReplicaSet

Apply pod.yaml, then a ReplicaSet — kill a pod and watch it self-heal

2

Roll out a Deployment

Update the image, watch a rolling update, then roll it back

3

Run a StatefulSet

Scale mysql-0..2, inspect stable per-pod DNS and PVCs

4

Schedule Jobs & CronJobs

Run db-migrate to completion; schedule nightly-backup

5

Add Init & Sidecar

Add a wait-for-db init container, then a log-shipper sidecar

6

Observe Lifecycle & Shutdown

Attach hooks, delete a pod, trace the shutdown sequence in events

Key Takeaways

Match the workload to the job — every one of today's twelve exists to solve one shape of problem

Pod

One or more containers, smallest unit

ReplicaSet

Keeps N identical pods alive

Deployment

Rolling updates for stateless apps

StatefulSet

Stable identity + storage per pod

DaemonSet

One pod on every node

Job

Run once to completion

CronJob

Jobs on a schedule

Init Container

Setup before the app starts

Sidecar

Helper running alongside the app

Lifecycle Hooks

postStart / preStop actions

Startup/Shutdown

Probes + graceful termination